Separating Language Orchestration from Deterministic Control in Agentic Meal Recommendation

Sai Teja Paladi, CSE 798 Independent Study

November 2026

Abstract

Large language models can translate natural-language requests into tool calls, but delegating a complete recommendation process to a model confounds language, tool-use, and recommendation errors. We study a hybrid BEACON meal workflow in which models interpret requests while deterministic components retain identity binding, Knapsack selection, nutrition calculation, and validation. A hosted agent first reproduces all six direct Knapsack plans, but is 28 times slower. On 30 requests and five local models, an identity gate raises complete passes from 49/150 to 73/150 and reduces unsafe unclear-persona executions from ten to zero without improving valid tool trajectories; a no-LLM pipeline passes 30/30, showing that this request set does not justify model use. In direct meal selection, Knapsack produces 6/6 safe plans with mean Beacon quality 0.860; Granite produces 6/6 safe plans at 0.679, while Llama produces only 3/6 safe plans despite 0.906 mean quality on that safe subset. Finally, a 42-request study separates profile retrieval, recommendation, nutrition analysis, and violation checking into four tools. The deterministic baseline passes 38/42. Across 126 model-request traces, prompt-only agents pass 62 cases and release all 18 injected faulty plans; paired deterministic release enforcement raises passes to 80 and reduces unsafe releases to zero without changing model traces. The results support using models only for language-dependent interaction that outperforms a declared no-LLM baseline, while deterministic code retains constraint-sensitive computation and release control.

Keywords: LLM agents, tool use, meal recommendation, deterministic optimization, policy enforcement, clarification

1. Introduction

Meal recommendation combines two different kinds of work. Natural-language requests may be incomplete, ambiguous, or ask the system to ignore stored restrictions, while profile retrieval, constraint enforcement, and optimization should be reproducible. An LLM can be useful for the first kind of work, but it is not automatically the appropriate component for the second. Garimella, Srivastava, and Sheth show this distinction clearly for exact arithmetic: deterministic implementations achieve perfect accuracy with far lower latency than LLM and agentic alternatives [3]. Their architectural decision framework motivates using a model only where language understanding is useful and retaining deterministic procedures for exact work.

This paper applies that principle to the BEACON meal-recommendation setting. BEACON studies the tradeoff between nutrition and convenience for configurable, long-horizon meal recommendations [11, 12]. We do not replace or modify its Knapsack recommender. Instead, we investigate whether an LLM can reliably orchestrate the workflow around it and what happens when the user's persona is missing, unknown, or ambiguous.

The study asks five research questions:

- **RQ1:** Can one hosted LLM complete the required tool trajectory while preserving the exact output of direct deterministic Knapsack execution?
- **RQ2:** How robust is prompt-only orchestration across natural-language, restriction-override, missing, unknown, and ambiguous-persona requests?
- **RQ3:** Can a deterministic identity gate improve clarification safety without changing the recommender?
- **RQ4:** How do direct LLM-selected meal plans compare with deterministic Knapsack plans on constraint adherence, quality, nutrition fit, variety, and latency?
- **RQ5:** Can responsibility-based tools support multiple user intents, and does deterministic validation prevent a corrupted plan from reaching the response boundary?

The paper makes five bounded contributions:

1. a controlled separation between language-model orchestration and deterministic meal selection in the BEACON system.
2. an identity-gated hybrid architecture that resolves a persona outside the model, constrains identity-dependent tool calls, and renders verified results without a final generative step.
3. a trace-based, five-model evaluation that measures exact plan fidelity, tool order, clarification, unsafe execution, failures, and latency without an LLM judge.
4. a same-persona, same-catalog comparison between deterministic Knapsack and two local LLM-only selectors, scored after generation by the same Beacon evaluators.
5. a four-tool, multi-intent extension with deterministic nutrition analysis, violation checking, injected-fault evaluation, and a matched release-gate ablation across three local models.

This is not a claim of a new optimization algorithm, a new language model, or a clinically validated recommender. It is a controlled systems study of agentic orchestration around an existing deterministic recommender.

2. Related Work

2.1 Reasoning, acting, and recommendation agents

ReAct formalizes an agent pattern in which a language model interleaves reasoning, actions, and observations from an external environment [1]. Our implementation follows the observable part of this pattern: the model receives tool schemas, chooses tool names and arguments, observes structured results, and decides whether to call another tool or stop. The presence of a particular agent framework is not what makes this loop agentic.

RecMind applies an autonomous LLM agent to recommendation using planning, personalized memory, world knowledge, and tools [2]. It demonstrates that LLMs can add flexible language understanding and planning to recommendation tasks. Our study asks a different and narrower question. The LLM does not rank meals; it must correctly invoke an existing recommender. This boundary allows an orchestration failure to be separated from a meal-selection failure.

2.2 Choosing between deterministic and agentic computation

Doing Less with More compares native functions, service architectures, direct LLM responses, and tool-using agents on 1,000 exact expressions [3]. Its central lesson is not that agents are universally ineffective, but that architecture should follow task properties. Exact known procedures should remain deterministic, while models can be reserved for linguistic or uncertain steps. Our architecture transfers this idea from arithmetic to meal-agent workflow control: Knapsack selection and identity resolution remain deterministic; request interpretation and valid tool sequencing remain model-driven.

2.3 Evaluating tool-using agents

Tool-use evaluation requires more than judging final prose. τ-bench evaluates tool-agent-user interaction using domain policies and final environment state [4]. ToolSandbox emphasizes state dependencies, canonicalization, and insufficient-information cases [5]. The Berkeley Function-Calling Leaderboard also evaluates tool selection, arguments, multi-turn execution, and irrelevant or hallucinated calls [6]. Following this line of work, our evaluator checks the executed persona, tool order, Knapsack action, and exact returned plan rather than using an LLM to rate the response text.

The Agentic AI Test Checklist collects 110 implementation-oriented checks for terminal state, dependency ordering, tool allowlists, runtime enforcement, and related risks [15]. It is not a meal-recommendation benchmark, so we do not claim a score over the full checklist. We adapt selected checks into task-specific oracles: all postconditions must hold together, dependent tools must follow successful prerequisites, the tool set and turn count are bounded, and plan release is enforced by the application rather than only by a prompt.

2.4 Clarification and runtime policy enforcement

Clarification is a separate capability from tool invocation. Testoni and Fernández show that model uncertainty does not necessarily match human clarification behavior [7]. Zhang and Choi likewise frame deciding when to ask as an

explicit stage in ambiguity resolution [8]. These findings motivate treating missing or ambiguous identity as an observable test condition rather than assuming that a system prompt will reliably produce a question.

AgentDojo shows that tool-using agents may fail tasks and security properties even without or under prompt-injection attacks [9]. CaMeL proposes placing a protective system layer around the model and enforcing capabilities through explicit control and data flows [10]. Our policy gate is substantially smaller and does not inherit CaMeL's formal guarantees. It adopts the relevant design principle: a consequential identity-binding decision should be enforced by the application rather than delegated solely to prompt compliance. The current prototype performs identity resolution, not authentication: it does not verify that the requester is entitled to access the named profile.

2.5 BEACON and meal recommendation

BEACON formulates meal planning as a combination problem involving food roles, user constraints, convenience, nutrition, and longer-term considerations [11, 12]. Its prototype includes deterministic recommendation methods and quality measures. Our study reuses its personas, local food sources, Knapsack implementation, and evaluators. We hold these components fixed so the experiment measures orchestration rather than introducing a competing meal algorithm.

2.6 MCP and LangChain

The Model Context Protocol standardizes how AI applications discover and invoke external tools and resources [13]. LangChain provides an agent harness that can implement a model-tool loop [14]. Neither changes the scientific distinction between model-controlled orchestration and deterministic selection. The present benchmark therefore uses a small native OpenAI-compatible function-calling loop, which makes every request, response, and tool trace directly auditable. An MCP or LangChain implementation would be a useful portability or framework-overhead comparison, but it is not the novelty claimed here.

3. System Design

3.1 Fixed recommendation component

For a stored persona p, the BEACON Knapsack implementation produces a one-day plan:

$$K(p, C, d) \to M,$$

where C is the fixed local catalog, d is the fixed planning date, and M contains breakfast, lunch, and dinner role-item assignments. The same persona, catalog, date, and backend version are used in all conditions.

The language model never supplies meal IDs to Knapsack and never replaces the selected plan. Consequently, if two conditions successfully execute Knapsack for the same persona, their expected role-item assignments are identical.

3.2 Agent tools

The agent receives three logical tools:

Tool	Purpose	State effect
`list_personas()`	Return stored persona identifiers and names.	Read only
`get_persona(persona_id)`	Retrieve one stored profile.	Records retrieved identity
`run_knapsack_for_persona(persona_id)`	Execute BEACON Knapsack.	Produces the meal plan

For valid requests, the required trajectory is:

```
user request
    → model calls list_personas
    → model calls get_persona for the intended persona
    → model calls run_knapsack_for_persona for the same persona
    → verified deterministic plan
```

The model genuinely chooses each tool and its arguments. A wrong tool, wrong persona, wrong order, missing call, or repeated execution is preserved in the trace and scored as a failure.

3.3 Prompt-only architecture

In the prompt-only condition, the system prompt tells the model to identify the persona, retrieve the profile, preserve stored restrictions, ask for clarification when necessary, and invoke Knapsack. These instructions are not enforced outside the model. The model can therefore choose an unrelated persona, skip a step, provide the full name where an internal identifier is required, or run Knapsack despite insufficient information.

3.4 Identity-gated architecture

The proposed architecture adds four deterministic controls:

1. **Identity resolution.** Before model execution, application code performs a case- and punctuation-insensitive match against one explicit full name or unique first name in the stored persona set.
2. **Safe clarification.** Missing, unknown, or multiple persona matches return a deterministic clarification. The model and identity-dependent tools are not called.
3. **Argument restriction.** For a resolved request, `get_persona` and `run_knapsack_for_persona` are locked to the resolved identifier. Knapsack also requires the list and matching-profile steps to have succeeded.
4. **Trusted rendering.** After successful Knapsack execution, the application returns a short confirmation from trusted state. It does not ask the model to restate nutrition facts or invent an explanation.

This gate does not choose meals and does not complete the valid tool sequence for the model. It converts incorrect model actions into explicit failures while preventing them from becoming wrong-person recommendations.

3.5 Responsibility-based tool extension

The initial three tools expose implementation steps rather than all of the responsibilities discussed for a meal agent. The extension uses four tools with distinct inputs, outputs, and failure boundaries:

Tool	Responsibility	Enforced prerequisite
`fetch_persona_profile`	Resolve one ID, full name, or unique first name and return the stored fictional profile.	Application-resolved persona must match.
`recommend_meal_plan`	Run the unchanged deterministic Beacon Knapsack method.	Matching profile was fetched.
`calculate_nutrition_scores`	Return nutrient totals, goal fit, and FSA/WHO-style proxies when requested.	A current plan exists.
`check_plan_violations`	Detect structural and stored dietary-condition violations.	A current plan exists.

Profile selection and retrieval are combined because they constitute one read-only database access responsibility. Recommendation, nutrition analysis, and validation remain separate because they answer different intents and fail independently. Plans are held in trusted session state, so the model does not copy a large plan into the nutrition or validation tools.

For a resolved request, the model still chooses which tools to invoke. The application binds all calls to the resolved profile and enforces dependencies. In the policy-gated condition, it also refuses to release a plan unless the violation checker has passed. A prompt-only ablation disables only this final release rule; it does not change the prompt, schemas, model, or recommender.

The FSA-style value is the mean of available per-item traffic-light proxies using sugar, salt converted from sodium, fat, and saturated fat (range 4--12; lower is better). The WHO-style value is the mean of available custom per-item range checks over seven nutrients (range 0--7; higher is better). Coverage is reported and missing values are not imputed. Neither proxy is an official label, clinical score, or individualized advice.

For example, Earl's deterministic plan contains nine selected catalog entries, but only five have the complete fields required by both proxies. The tool therefore returns FSA-style 9.8 and WHO-style 3.8 with coverage 5/9, rather than silently filling the four missing entries. Its nutrition-goal fit is 0.506: the calorie component is 1.000, while carbohydrate, protein, and fiber fits are 0.307, 0.530, and 0.187. Nutrient totals sum available catalog values and must be read together with the missing-data fields.

4. Experimental Method

4.1 Experiment 1: direct versus agentic integration check

Experiment 1 is an integration and tool-compliance check for RQ1 using the six fictional BEACON personas. The direct condition receives a selected persona identifier and invokes Knapsack. The agentic condition receives a natural-language request, uses hosted `openai/gpt-oss-20b` through Groq at temperature zero, and must complete the three-tool trajectory. Both conditions use the same backend, catalog, fixed date, and reference plans. Because Knapsack is deterministic, identical meal output is expected whenever the model supplies the correct persona identifier; the nontrivial measurement is whether this one hosted model completes the required tool trajectory.

The metrics are requested-role coverage, exact role-item fidelity to direct Knapsack, required tool-sequence compliance, BEACON's original goodness, BEACON's newer quality score, post-hoc nutrition-goal fit, failures, retries, and end-to-end latency. Nutrition fit is evaluated after generation; the tested Knapsack implementation does not optimize the numeric calorie, carbohydrate, protein, and fiber targets directly.

4.2 Experiment 2: request robustness

Experiment 2 tests RQ2 and RQ3 with 30 fixed requests over the same six stored personas. Twenty-four requests require recommendation: six each use an exact full name, a paraphrase, a unique first name, or a restriction-override request that asks the system to ignore saved restrictions. Six requests require clarification because identity is unknown, missing, or ambiguous.

Five local Ollama models are evaluated:

- `llama3.2:1b`;
- `llama3.2:3b`;
- `phi4-mini:3.8b`;
- `qwen3.5:4b`;
- `granite4:3b`.

Each model receives every request in both the prompt-only and identity-gated architectures, producing 300 case-level executions. A no-LLM pipeline also applies the same name resolver and then runs Knapsack directly. Runs are sequential to avoid local memory contention. Temperature is zero, the turn limit is eight, and the same tool schemas, backend, reference plans, and scoring rules are used.

4.3 Experiment 3: deterministic versus LLM-only selection

Experiment 3 tests RQ4 using the same six personas and local Beacon catalog. The deterministic condition runs Knapsack. In each LLM-only condition, the model receives the stored profile and all 124 unique candidates eligible for the requested main-course, side, or beverage roles (145 role memberships) in a compact representation containing candidate ID, name, role, dietary flags, and available calories, carbohydrates, protein, and fiber. The LLM selects the nine catalog items directly. It cannot call Knapsack or another recommendation tool, and candidates are not pre-filtered for the persona's dietary conditions.

To measure meal choice rather than malformed ID copying, Ollama structured output constrains each field to an ID belonging to the requested role and catalog. This schema performs no ranking and applies no persona-specific dietary filter. `granite4:3b` and `llama3.2:3b` each make one call per persona at temperature zero. After selection, deterministic validation checks stored dietary conditions, and the same Beacon evaluators measure goodness, overall quality, nutrition fit, repeated foods, exact Knapsack match, tokens, and latency.

4.4 Experiment 4: meaningful tools and fault containment

Experiment 4 evaluates the four responsibility-based tools on 42 fixed non-sensitive requests, six in each of seven categories: profile-only, recommendation, nutrition analysis, restriction override, indirect intent, unclear identity, and injected fault. Unclear identities are handled before model execution by the same deterministic resolver. For every other case, the model must select an intent-appropriate sequence rather than replay one fixed trajectory.

The injected fault removes the first required role from a completed plan after Knapsack and before validation. This isolates whether the checker detects the fault and whether the release boundary enforces that result. It does not test

whether the model can notice the corruption from text because the model never receives or edits the plan directly.

Three local Ollama models are used: `granite4:3b`, `qwen3.5:4b`, and `llama3.2:3b`. Each model receives all 42 requests once with the release gate disabled. To isolate the gate from model nondeterminism, each of the 126 model-request traces is then replayed through the deterministic gate without another model call. A deterministic keyword pipeline runs the same 42 cases as a no-LLM baseline. All runs use the same dataset, four tool schemas, backend, Knapsack implementation, temperature zero, and eight-turn limit.

4.5 Scoring

A valid request passes only if it:

1. retrieves the intended profile;
2. invokes Knapsack exactly once for that persona;
3. follows `list → profile → Knapsack`;
4. exactly matches the direct Knapsack role-item assignments.

An unclear request passes only if it returns a usable clarification and does not run Knapsack. We separately count unsafe Knapsack executions, policy-blocked calls, API or loop failures, failed tools, and wall-clock latency. No generated text is graded for recommendation quality by another model.

An LLM-only plan is safe and valid only if it fills every requested meal role with a catalog member and violates none of the stored diabetes, vegetarian, vegan, or gluten-free conditions. Quality metrics are computed after generation and do not influence the model's selection.

For Experiment 4, complete-case success requires all applicable checks to pass: persona routing, exact tool membership with dependency-valid order, expected terminal action, correct nutrition-tool selection, exact Knapsack fidelity, zero failed tools, and zero API or loop errors. A fault case also requires detection and blocked release. We separately report unsafe releases and success by request category. The metric dictionary gives each denominator and a worked example.

4.6 Reproducibility

The benchmark records the dataset hash, prompt hash, source commit, hashes of the benchmark and policy-gate implementations, model, temperature, complete tool trace, application errors, and latency. All profiles are fictional and all food sources are local to the compatible BEACON checkout.

5. Results

5.1 Direct versus agentic integration check

Condition	Completed	Role coverage	Exact fidelity	Required sequence	Goodness	BEACON quality	Nutrition fit	Mean latency
Direct Knapsack	6/6	100%	100%	N/A	1.000	0.860	0.549	202.4 ms
Agentic Knapsack	6/6	100%	100%	100%	1.000	0.860	0.549	5706.4 ms

The hosted agent completes the required tool sequence and reproduces the deterministic output for all six personas. This is evidence of integration and tool-use compliance for one model, not evidence that the LLM discovers a better recommendation. Meal-quality metrics are identical because the same Knapsack method selects the food. The agentic path is approximately 28 times slower when provider retry waits are included. Excluding 23.8 seconds of recorded rate-limit waits, its mean latency is 1747.2 ms, still about 8.6 times the direct baseline.

5.2 Prompt-only, identity-gated, and no-LLM robustness

Model	Architecture	Overall	Valid requests	Unclear requests	Unsafe executions	Mean latency
None	No-LLM pipeline	**30/30**	**24/24**	**6/6**	**0**	**146.3 ms**
Granite 4 3B	Prompt-only	22/30	22/24	0/6	6	9815.1 ms
Granite 4 3B	Identity-gated	**28/30**	22/24	**6/6**	**0**	**3154.7 ms**
Llama 3.2 1B	Prompt-only	0/30	0/24	0/6	0	3596.2 ms
Llama 3.2 1B	Identity-gated	**6/30**	0/24	**6/6**	0	**2150.2 ms**
Llama 3.2 3B	Prompt-only	5/30	5/24	0/6	3	3868.7 ms
Llama 3.2 3B	Identity-gated	**11/30**	5/24	**6/6**	**0**	**2809.4 ms**
Phi-4 Mini 3.8B	Prompt-only	6/30	0/24	6/6	0	**1992.8 ms**
Phi-4 Mini 3.8B	Identity-gated	6/30	0/24	6/6	0	2001.4 ms
Qwen 3.5 4B	Prompt-only	16/30	16/24	0/6	1	15351.0 ms
Qwen 3.5 4B	Identity-gated	**22/30**	16/24	**6/6**	**0**	**8953.8 ms**

Aggregating the matched runs gives the following result:

Measure	Prompt-only	Identity-gated
Complete-case passes	49/150	**73/150**
Valid workflow passes	43/120	43/120
Unclear-request passes	6/30	**30/30**
Unsafe unclear-case Knapsack executions	10	**0**
Mean latency per case	6924.8 ms	**3813.9 ms**

The policy gate improves all 24 unclear cases that were not already complete passes and introduces no matched pass-to-fail regressions. The 44.9% mean latency reduction comes from bypassing model inference for unclear identity and omitting an unnecessary final model turn after verified Knapsack execution.

The no-LLM pipeline is both faster and more accurate on this dataset. Every valid request contains an exact full name or unique first name, every unclear case is covered by the resolver's rules, and the remaining workflow is fixed. Its 30/30 result is therefore close to true by construction. It establishes an important negative result: these 30 cases do not show that an LLM is needed. Requests involving typos, nicknames, contextual references, or free-form preferences are needed to test added language-understanding value.

The 43/120 valid-workflow total is identical because the same valid requests fail in matched prompt-only and gated runs. Across the five prompt-only runs, 43 cases pass, 41 fail in backend tool calls because the model supplies an invalid persona argument, and 36 never make a successful Knapsack call. In the gated runs, the 41 backend errors become 41 explicit policy blocks; the other 36 still lack a successful call. The gate contains invalid arguments but does not repair the model's action or force the sequence.

5.3 Deterministic versus LLM-only selection

Selector	Safe plans	Hard violations	Exact Knapsack matches	Goodness	Quality, all cases	Quality, safe only	Nutrition fit	Repeated foods	Latency
Beacon Knapsack	**6/6**	**0**	6/6 reference	**1.000**	**0.860**	0.860	0.549	**0.0**	**183.5 ms**
Granite 4 3B LLM-only	**6/6**	**0**	0/6	**1.000**	0.679	0.679	0.479	2.0	4338.9 ms
Llama 3.2 3B LLM-only	3/6	4	0/6	0.981	0.453	**0.906**	**0.563**	**0.0**	4035.3 ms

Granite returns six dietary-adherent plans, but its mean Beacon quality is 0.181 below Knapsack, nutrition fit is lower, and it repeats two foods per plan on average. Llama avoids repeated foods and slightly exceeds Knapsack's mean nutrition fit, but it violates stored dietary conditions for Jessica, Becca, and Dylan. The four violations are two non-low-sugar choices for a persona with diabetes and one meat-containing choice for each of two vegetarians. Beacon assigns zero overall quality to those unsafe plans, reducing Llama's mean quality to 0.453. Its three safe plans average 0.906, above Knapsack's 0.860. This conditional result is based on only three selected cases and must be read together with Llama's 3/6 safety rate; it prevents a claim that Knapsack has the highest conditional meal quality.

Neither model exactly reproduces a Knapsack plan. This demonstrates that the models are genuinely selecting different meals; exact disagreement alone is not an error. The safety and quality metrics show why the difference matters. Both LLMs are more than 20 times slower than direct Knapsack and consume about 12,900 prompt-plus-completion tokens across six personas, whereas Knapsack uses no model tokens.

5.4 Meaningful-tool and fault-containment result

Model or baseline	Prompt-only	Policy-gated paired replay	Valid tool path	Unsafe releases, prompt → gated	Mean latency
Deterministic rule pipeline	38/42	N/A	90.5%	0 → 0	264.9 ms
Granite 4 3B	24/42	**30/42**	71.4%	6 → **0**	4573.8 ms
Llama 3.2 3B	19/42	**25/42**	61.9%	6 → **0**	3630.6 ms
Qwen 3.5 4B	19/42	**25/42**	59.5%	6 → **0**	7843.7 ms

Across the same 126 model-request traces, the release gate increases complete passes from 62 to 80 and reduces unsafe release of injected faulty plans from 18 to zero. Every increase is a fault case whose model called the violation checker and detected the missing role but whose prompt-only application would still release the plan. Because each trace is replayed rather than regenerated, the gate cannot change identity routing, tool selection, Knapsack output, model failures, or latency.

The broader workflow remains difficult. Granite has 100% Knapsack fidelity over the 24 ordinary recommendation-required cases, but it completes only one of six plain recommendation cases and none of six nutrition-analysis cases because it usually omits the final violation check. Llama and Qwen have only 41.7% and 29.2% Knapsack fidelity respectively because they often stop before producing a plan or choose an invalid workflow. Llama records 27 failed local calls and two turn-limit failures; the other two models have no API or loop failures.

Category results also show why one overall score is insufficient. All three agents handle six of six profile-only requests, whereas the keyword baseline handles three. Granite passes all restriction-override requests, while Llama and Qwen pass none. Nutrition-analysis success is 0/6 for Granite, 2/6 for Llama, and 1/6 for Qwen. All unclear-identity cases pass because the application resolver clarifies before model execution; this is a system result, not model evidence. The deterministic pipeline's 38/42 remains the strongest overall result and is 13.7--29.6 times faster than the three agent runs.

5.5 Model capability versus architecture safety

The valid-workflow total is exactly 43/120 in both architectures. This is the most important interpretation boundary in the study: the gate does not improve the model's ability to select tools. Granite and Qwen remain substantially more capable than the smaller models on valid trajectories. The gate instead changes what the system permits when the model is wrong or the request is incomplete.

Phi-4 Mini illustrates the distinction. It safely clarifies all six unclear requests even without the gate but completes none of the 24 valid workflows. Granite illustrates the opposite failure: it completes 22/24 valid workflows but runs Knapsack on all six unclear prompt-only requests. General tool-use capability and safe clarification are therefore empirically separable.

6. Discussion

6.1 Answering the research questions

RQ1: One hosted model completes all six required tool trajectories and preserves the fixed Knapsack output. This is a small integration result. It adds substantial latency and does not test whether language understanding is useful.

RQ2: Prompt-only robustness varies widely by model. No tested model combines high valid-workflow completion with consistently safe unclear-persona handling. The strongest prompt-only model completes 22/24 valid cases but executes the recommender in all six cases where clarification is required.

RQ3: The deterministic gate produces complete safe clarification for all 30 unclear model-request pairs, eliminates ten unsafe executions, and reduces mean latency without changing valid-workflow performance. It improves the system, not the underlying models. The no-LLM baseline's 30/30 result further shows that the present request set does not require a model.

RQ4: In this preliminary controlled comparison, Granite preserves dietary constraints but has lower all-case quality, nutrition fit, and variety than Knapsack. Llama violates hard conditions in half of the personas, although its three safe plans have higher conditional quality. Knapsack is faster, token-free, and safe on all six cases. The small sample and conditional result do not support a general ranking of selector quality.

RQ5: Responsibility-based tools make profile retrieval, recommendation, nutrition analysis, and validation separately observable, but none of the three models matches the 38/42 deterministic baseline. The paired release gate turns all 18 detected injected faults into blocked outcomes without changing a tool trace. It improves system safety, not language understanding or tool-use capability.

6.2 What is novel

The novelty is the combination of a task-specific architecture and controlled evaluation:

- it isolates natural-language orchestration from deterministic meal selection;
- it converts persona resolution into an explicit application-level control decision instead of relying only on a system prompt;
- it evaluates the same agent with and without that gate across five local models and identical requests;
- it scores executed state—identity, tool order, Knapsack action, and exact plan fidelity—while separately reporting safety and model capability;
- it compares deterministic and direct LLM selection on the same profiles, catalog, roles, and post-hoc evaluators instead of comparing systems on incompatible inputs.
- it evaluates four responsibility-based tools on multiple intents and isolates the release gate's effect with trace-paired fault replay rather than separate model samples.

The work does not claim that deterministic identity resolution is itself a new idea. Its contribution is applying and measuring that principle in a reproducible BEACON meal-agent workflow, exposing a concrete tradeoff that a final-answer accuracy metric would hide: a model can be good at valid tool trajectories and simultaneously unsafe when identity is unclear.

6.3 Architecture lesson from direct LLM selection

The direct-selection comparison clarifies the architecture tradeoff. The LLM-only selectors are not merely slower alternatives that produce the same plans. They make different tradeoffs: Granite repeatedly chooses foods despite being safe, while Llama obtains a favorable numeric nutrition-fit mean partly through plans that violate stored dietary conditions. A single aggregate metric would hide both failures.

The result supports a hybrid design in which deterministic code first enforces hard constraints and produces feasible candidates. An LLM may still add value by reranking that bounded set using soft preferences or by explaining tradeoffs. This is different from allowing the model to select from the unrestricted catalog and hoping a prompt will preserve every condition.

7. Limitations and Ethical Scope

The evaluation uses six fictional personas and 30 handcrafted requests. Exact first-name matching is deliberately conservative and does not resolve typos, nicknames, pronouns, or conversational identity. The study uses one local machine and one run per model-request pair at temperature zero; it does not estimate variance across sampling, hardware, or model versions. The models are small local models, so results should not be generalized to frontier hosted systems.

The responsibility-based extension adds 42 handcrafted cases but uses the same six personas, one prompt per category and persona, and three small local models. Its unclear-identity outcomes come from the pre-model resolver and do not test model clarification. Its fault set contains one synthetic corruption—removing the first required meal role—so zero unsafe releases demonstrates enforcement for that fault, not complete safety. The paired replay isolates the gate's effect, but it does not estimate variation across repeated model generations.

The gate's six unclear cases were handcrafted for the resolver's fixed rules, so its perfect clarification result is an invariant on this test set rather than evidence of generalization. Name matching is identity resolution, not authentication: anyone can type another persona's name. A stronger prompt-only baseline with few-shot examples

or an explicit `ask_clarification` tool was not tested. Those baselines, repeated trials, and language-dependent requests are required before claiming that the gate is preferable to the strongest prompting or that an LLM adds value over the no-LLM pipeline.

The LLM-only experiment uses role- and catalog-constrained JSON generation. It therefore measures meal choice after eliminating malformed role/ID output; it does not measure fully unconstrained generation. It does not filter candidates by persona, so the observed dietary violations remain model decisions. Only two models are included in this recommendation-source comparison.

Meal plans are evaluated using existing BEACON metrics and deterministic fidelity. The study does not establish that plans are clinically appropriate, that users will follow them, or that the policy gate provides complete security. Nutrition-goal fit is post-hoc and must not be described as an optimized constraint. No real personal or protected health information is used. Because Knapsack and the BEACON evaluators come from the same system, the quality measures are not an independent human judgment and may align with the deterministic method's design objectives.

The FSA-style and WHO-style outputs are application-defined per-serving proxies, not official labels. In the worked plan, only five of nine selected entries have complete inputs, and nutrient totals can be partial when catalog fields are missing. Coverage and missing-data fields must accompany every reported value.

The aggregate 49/150 and 73/150 observations contain repeated prompts across models and should not be treated as 150 independent human samples. The current result is a controlled systems comparison, not a user study or clinical trial.

8. Next Experiment: Constrained Hybrid Reranking

The next study should add a hybrid selection condition while retaining the two completed baselines. It should hold personas, catalog, candidate visibility, and evaluation metrics fixed and compare:

1. **Deterministic Knapsack:** the existing reference selector;
2. **LLM-only selector:** the completed direct-selection condition;
3. **Constrained hybrid reranker:** deterministic code produces several feasible candidate plans, the LLM reranks them for stated soft preferences, and a deterministic validator rejects any invalid output.

Run each stochastic condition repeatedly and expand the persona/request set. Primary measures should include hard-constraint violations, BEACON goodness, quality and nutrition-goal fit, preference satisfaction, exact consistency over repeated runs, explanation faithfulness, latency, and token cost. This three-way experiment would answer whether language-model judgment adds recommendation value. It must remain separate from the orchestration experiment so a ranking change is not confused with a routing failure.

9. Conclusion

This study demonstrates that an LLM can correctly orchestrate an existing deterministic meal recommender, but success on clear requests does not ensure safe behavior when identity is insufficient. Prompt-only agents either fail valid workflows, fail clarification, or both. A small deterministic policy gate eliminates observed wrong-person Knapsack executions and reduces latency while preserving the models' valid-request performance. However, the no-LLM pipeline solves the current 30-case request set faster and without model calls, so the present experiment does not establish a need for an LLM. The main lesson is a division of responsibility. The direct-selection comparison reinforces this conclusion: one LLM sacrifices quality and variety while the other violates stored dietary conditions despite a favorable nutrition-fit mean. The four-tool extension adds a second lesson: all three models detect the injected fault when they reach validation, but a deterministic release boundary is required to ensure that the detected fault is actually blocked. Use the model for language-dependent orchestration only when a harder language benchmark shows a benefit, and for bounded soft-preference ranking; retain constraint-sensitive recommendation and identity-binding decisions in inspectable deterministic components; and evaluate both the trajectory and the resulting state.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*, 2023.

[2] Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang. "RecMind: Large Language Model Powered Agent for Recommendation." *Findings of NAACL*, pp. 4351–4364, 2024. doi:10.18653/v1/2024.findings-naacl.271.

[3] R. Garimella, B. Srivastava, and A. Sheth. "Doing Less with More: A First-Principles Exploration of the Suitability of Agentic Computing over Alternative Architectural Choices." 2026.

[4] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." 2024.

[5] J. Lu et al. "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities." 2024.

[6] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez. "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models." *Proceedings of the 42nd International Conference on Machine Learning*, pp. 48371–48392, 2025.

[7] A. Testoni and R. Fernández. "Asking the Right Question at the Right Time: Human and Model Uncertainty Guidance to Ask Clarification Questions." *EACL*, pp. 258–275, 2024. doi:10.18653/v1/2024.eacl-long.16.

[8] M. J. Q. Zhang and E. Choi. "Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs." *Findings of NAACL*, pp. 5541–5558, 2025. doi:10.18653/v1/2025.findings-naacl.306.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." 2024.

[10] E. Debenedetti et al. "Defeating Prompt Injections by Design." 2025.

[11] V. Nagpal, S. L. Valluru, K. Lakkaraju, and B. Srivastava. "BEACON: Balancing Convenience and Nutrition in Meals With Long-Term Group Recommendations and Reasoning on Multimodal Recipes." 2024.

[12] V. Nagpal, S. L. Valluru, K. Lakkaraju, N. Gupta, Z. Abdulrahman, A. Davison, and B. Srivastava. "A Novel Approach to Balance Convenience and Nutrition in Meals With Long-Term Group Recommendations and Reasoning on Multimodal Recipes and its Implementation in BEACON." 2024.

[13] Model Context Protocol. "Specification." Accessed October 27, 2026.

[14] LangChain. "Agents." Accessed October 27, 2026.

[15] S. Padmanabhuni. "Agentic AI Test Checklist 110." Hugging Face dataset, 110 rows, CC BY 4.0. Accessed November 3, 2026.